AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm drzewa BST
z zastosowaniem GitHub

Autor:
Jakub Pyrdoł
Tymoteusz Zaczyk
Dawid Plewa

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**
1.1. Przykład 4
1.2. Instalacja 5
1.3. Wykorzystanie Git i GitHub 6
2. Analiza problemu **7**
2.1. Zastosowanie i reprezentacja danych 7
2.2. Sposób działania algorytmu 7
2.3. Ręczny przykład działania 8
2.4. Git i GitHub w projekcie 9
Literatura **10**
Spis rysunków **10**
Spis tabel **10**
Spis listingów **10**

1. Ogólne określenie wymagań

Celem projektu jest przygotowanie binarnego drzewa poszukiwań (BST, ang. *Binary Search Tree*) przechowującego liczby całkowite. Program ma przedstawiać sposób organizowania danych w strukturze drzewiastej oraz umożliwiać dodawanie i usuwanie elementów z zachowaniem określonego porządku. Projekt obejmuje również wykorzystanie systemu kontroli wersji Git oraz platformy GitHub do przechowywania kodu i zapisywania kolejnych etapów jego rozwoju.

Drzewo składa się z węzłów połączonych relacjami rodzic–dziecko. Węzeł rozpoczynający strukturę nazywany jest korzeniem. Każdy węzeł drzewa binarnego może mieć najwyżej dwoje dzieci: lewe i prawe. Węzeł bez dzieci jest liściem, natomiast węzły znajdujące się poniżej danego elementu tworzą jego poddrzewa. Taki sposób organizacji danych pozwala opisywać zależności pomiędzy elementami bez ograniczania ich do jednej kolejności liniowej.

W drzewie BST obowiązuje dodatkowa reguła porządku. Wszystkie wartości lewego poddrzewa są mniejsze od wartości rozpatrywanego węzła, a wszystkie wartości prawego poddrzewa są większe. Ta sama zasada obowiązuje na każdym poziomie drzewa. Dzięki temu porównanie wartości pozwala wybrać dalszy kierunek przejścia: w lewo albo w prawo. W bieżącym projekcie jednakowe wartości nie są przechowywane wielokrotnie.

Drzewa poszukiwań są przydatne przy organizowaniu zbiorów kluczy oraz danych, dla których istotne są porównania i relacja porządku. Praca nad taką strukturą pozwala poznać zależność pomiędzy przyjętym sposobem przechowywania danych a działaniem operacji wykonywanych na tych danych. W tym projekcie istotne jest zarówno poprawne utworzenie połączeń, jak i ich zachowanie po dodaniu lub usunięciu elementu.

Węzły są przechowywane w kontenerze `std::vector<Node>`. Każdy z nich zawiera liczbę całkowitą oraz indeksy lewego i prawego dziecka. Połączenia nie są więc zapisane za pomocą wskaźników do oddzielnych obiektów. Pozycja elementu w wektorze określa miejsce przechowywania, natomiast zapisane indeksy określają jego miejsce w logicznej strukturze drzewa.

Dokumentacja dotyczy bieżącego stanu kodu. W obecnej wersji dostępne są:

- utworzenie pustego drzewa, do którego można następnie dodawać kolejne wartości;

- dodawanie elementów we właściwym miejscu drzewa z pomijaniem wartości już występujących;
- usuwanie węzła o wskazanej wartości z aktualizacją połączeń pomiędzy pozostałymi elementami;
- opróżnianie całego drzewa i ponowne wykorzystanie tego samego obiektu;
- wyświetlanie wartości w kolejności ich przechowywania w kontenerze.

Program działa w konsoli i wykonuje ustalony scenariusz z funkcji `main()`. Poszczególne operacje są wywoływane bezpośrednio w kodzie, a ich efekty przedstawiane w postaci komunikatów i wypisywanych wartości. Projekt nie ma obecnie menu do wprowadzania danych. Metoda `show()` wypisuje kolejność w wektorze; nie realizuje przejść preorder, inorder ani postorder.

Wykorzystanie GitHuba stanowi drugi element projektu, obok samego algorytmu drzewa. Pozwala prowadzić rozwój kodu w repozytorium i zachować historię wprowadzanych zmian. Rolę Git oraz GitHuba w projekcie opisano w podrozdziale 1.3.

1.1. Przykład

Przykład zawarty w funkcji `main()` pokazuje sposób korzystania z przygotowanej klasy drzewa. Najpierw tworzony jest pusty obiekt, a następnie dodawane są do niego wartości 1, 2, 3, 8 oraz 5. Fragment odpowiedzialny za dodawanie danych przedstawia listing 1.

```
8     bst.addNode(1);
9     bst.addNode(2);
10    bst.addNode(3);
11    bst.addNode(8);
12    bst.addNode(5);
```

Listing 1. Dodawanie przykładowych wartości w funkcji main

Pierwsza wartość staje się korzeniem. Kolejne liczby są porównywane z wartościami już znajdującymi się w drzewie i kierowane do odpowiedniego poddrzewa. Liczby 2, 3 i 8 tworzą w tym przykładzie ciąg prawych połączeń. Wartość 5 zostaje natomiast umieszczona jako lewe dziecko węzła 8. Dobór danych pozwala więc przedstawić oba kierunki połączeń.

Po dodaniu elementów program wyświetla zawartość kontenera. Wynik ma postać $1, 2, 3, 8, 5$, ponieważ metoda `show()` odczytuje kolejne pozycje wektora. Nie jest to wypisywanie wartości w porządku rosnącym.

W dalszej części programu usuwana jest wartość 3. Po tej operacji wyświetlane są wartości $1, 2, 8, 5$. Następnie całe drzewo jest opróżniane, a do tego samego obiektu dodawana jest wartość 42. Ostatni etap pokazuje możliwość ponownego użycia obiektu po usunięciu jego wcześniejszej zawartości. Ręczny przebieg usunięcia wartości 3, wraz ze zmianą połączeń i indeksów, opisano w podrozdziale 2.3.

1.2. Instalacja

Do kompilacji wymagany jest kompilator obsługujący standard C++14. Program korzysta z biblioteki standardowej C++, w szczególności z kontenera `vector` oraz strumieni `iostream`. Nie wymaga instalowania dodatkowych bibliotek. Za przygotowanie budowania odpowiada plik `CMakeLists.txt`, który określa nazwę celu `Drzewo_BST` i wymaga CMake w wersji co najmniej 4.1.

Kod można pobrać z GitHuba. Adres repozytorium podano w podrozdziale 1.3. Można pobrać archiwum ZIP albo skorzystać z lokalnej kopii repozytorium Git.

Po rozpakowaniu należy zachować wspólny katalog plików `main.cpp`, `BST.cpp`, `BST.h` oraz `CMakeLists.txt`. Pliki te zawierają program główny, definicję drzewa i konfigurację jego kompilacji.

W katalogu źródeł należy otworzyć terminal i przygotować katalog budowania poleceniem:

```
cmake -S . -B build
```

Opcja `-S .` wskazuje bieżący katalog jako miejsce przechowywania źródeł, natomiast `-B build` określa katalog, w którym CMake umieści pliki związane z budowaniem aplikacji. Pozwala to oddzielić źródła od wyników kompilacji.

Następnie należy zbudować program:

```
cmake --build build
```

Polecenie korzysta z wcześniej przygotowanej konfiguracji i uruchamia właściwe narzędzie budowania. Powstały plik `Drzewo_BST`, a w systemie Windows zwykle `Drzewo_BST.exe`, znajduje się w katalogu budowania lub jego podkatalogu zależnym od generatora i konfiguracji.

Po uruchomieniu aplikacja wykonuje scenariusz zapisany w `main.cpp` i wypisuje wyniki w konsoli. Aby użyć innego zestawu liczb lub zmienić kolejność operacji, należy zmodyfikować tę funkcję, zapisać plik i ponownie wykonać polecenie budowania. Samo pobranie kodu z GitHuba nie zastępuje kompilacji programu na komputerze użytkownika.

1.3. Wykorzystanie Git i GitHub

Projekt jest rozwijany z wykorzystaniem Git oraz platformy GitHub. Repozytorium kodu znajduje się pod adresem:

https://github.com/pyr-jak/Drzewo-BST

Git jest rozproszonym systemem kontroli wersji. Pozwala zapisywać zmiany w plikach w postaci commitów, czyli kolejnych zatwierdzonych wersji projektu. Historia tych zapisów umożliwia porównanie zmian i odwołanie się do wcześniejszego stanu kodu.

GitHub udostępnia zdalne miejsce przechowywania repozytorium Git oraz narzędzia do przeglądania kodu, historii zmian i współpracy. Git odpowiada za wersjonowanie, a GitHub umożliwia udostępnienie repozytorium przez Internet.[1]

Podczas pracy nad drzewem BST kolejne zmiany można zapisywać w oddzielnych commitach i przesyłać do repozytorium zdalnego. Pozwala to powiązać zapisane wersje z rozwojem poszczególnych operacji drzewa.

Takie prowadzenie projektu porządkuje pracę i ogranicza potrzebę tworzenia wielu ręcznie nazwanych kopii plików. Ułatwia również udostępnienie kodu do oceny oraz dalszą rozbudowę programu z zachowaniem historii wcześniejszych rozwiązań.

[1] Rozróżnienie Git i GitHub opisano w oficjalnej dokumentacji: https://docs.github.com/en/get-started/start-your-journey/what-is-github.

2. Analiza problemu

2.1. Zastosowanie i reprezentacja danych

Drzewo BST służy do porządkowania danych według klucza. Porównanie z wartością bieżącego węzła pozwala wybrać jedno z poddrzew, zamiast sprawdzać wszystkie elementy. Struktura ta może być wykorzystywana do przechowywania zbiorów unikatowych wartości i budowy prostych indeksów.

W projekcie węzeł `Node` zawiera wartość `value` oraz pola `left` i `right`. Pola dzieci są indeksami w wektorze `tree`; wartość `-1` oznacza brak dziecka. Indeks korzenia znajduje się w `rootIndex`. Dla pustego drzewa również wynosi on `-1`.

Rozmieszczenie węzłów w wektorze jest oddzielone od ich połączeń. Element może leżeć pod dowolnym poprawnym indeksem, o ile pola dzieci prowadzą do właściwych poddrzew. Sama kolejność wektora nie musi być posortowana.

2.2. Sposób działania algorytmu

Dodawanie rozpoczyna się od korzenia. Wartość mniejsza jest kierowana do lewego poddrzewa, a większa do prawego. Po dojściu do pustego połączenia nowy węzeł zostaje dopisany na końcu wektora, a jego indeks zapisany jako połączenie od rodzica. Jeżeli wartość już występuje, operacja nie dodaje duplikatu.

Usuwanie rozpoczyna się od wyszukania wartości i zapamiętania rodzica znalezionego węzła. Gdy wartości nie ma, drzewo pozostaje bez zmian. Dla znalezionego elementu rozpatrywane są trzy przypadki:

- liść zostaje odłączony od rodzica;
- węzeł z jednym dzieckiem zostaje zastąpiony połączeniem do tego dziecka;
- węzeł z dwojgiem dzieci otrzymuje wartość minimum prawego poddrzewa, po czym usuwany jest węzeł tego następnika.

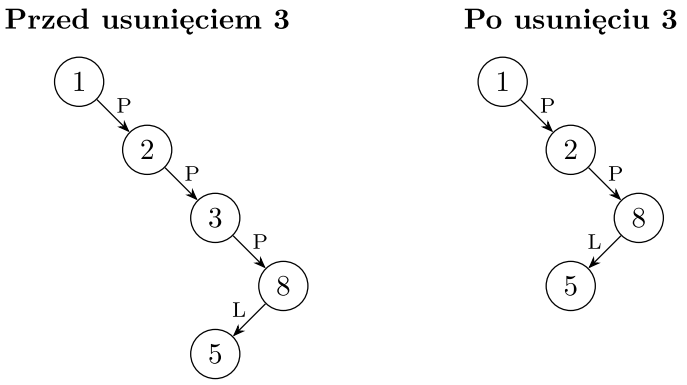
Usunięcie pozycji wektora przesuwa późniejsze elementy. Program zmniejsza więc o jeden wszystkie indeksy większe od usuniętej pozycji, uwzględniając również indeks korzenia. Metoda `removeTree()` czyści cały wektor i ustawia `rootIndex` na `-1`. Obiekt może następnie przyjmować kolejne wartości.

Dla wysokości drzewa h przejście po jednej ścieżce ma koszt $O(h)$. Drzewo nie jest równoważone, więc przy niekorzystnej kolejności danych wysokość może rosnąć liniowo

z liczbą węzłów n. Skuteczne usunięcie ma koszt $O(n)$ ze względu na przesuwanie elementów i poprawianie indeksów. Realokacja bufora wektora może również nadać pojedynczemu dodaniu koszt $O(n)$.

2.3. Ręczny przykład działania

Po dodaniu wartości $1, 2, 3, 8, 5$ korzeniem jest 1. Węzły 2, 3 i 8 tworzą ciąg prawych połączeń, natomiast 5 jest lewym dzieckiem węzła 8. Rysunek 2.1 pokazuje tę strukturę oraz wynik usunięcia wartości 3. Litery L i P oznaczają lewe i prawe dziecko.



Rys. 2.1. Drzewo przed usunięciem wartości 3 i po jej usunięciu

Węzeł o wartości 3 ma jedno dziecko: węzeł 8. Program łączy więc rodzica, czyli węzeł 2, bezpośrednio z węzłem 8. Następnie usuwa pozycję o indeksie 2 z wektora. Węzły 8 i 5 przesuwają się pod indeksy 2 i 3, dlatego trzeba poprawić prowadzące do nich połączenia.

Tabela 2.1 przedstawia zawartość wektora przed operacją i po korekcie indeksów. Korzeń nadal ma indeks 0. Kreski w ostatnim wierszu oznaczają pozycję, której po usunięciu nie ma już w kontenerze.

Tab. 2.1. Zmiana zawartości wektora po usunięciu wartości 3

	Przed usunięciem			**Po usunięciu**		
Indeks	**value**	**left**	**right**	**value**	**left**	**right**
0	1	−1	1	1	−1	1
1	2	−1	2	2	−1	2
2	3	−1	3	8	3	−1
3	8	4	−1	5	−1	−1
4	5	−1	−1	—	—	—

Po tej operacji `show()` wypisuje $1, 2, 8, 5$, zgodnie z kolejnością w wektorze. Nie jest to wynik przejścia inorder, które dla tego drzewa dałoby $1, 2, 5, 8$.

2.4. Git i GitHub w projekcie

Do wersjonowania projektu wykorzystywany jest Git, a repozytorium znajduje się pod adresem https://github.com/pyr-jak/Drzewo-BST [1]. Git pozwala zapisywać kolejne wersje plików i porównywać zmiany, natomiast GitHub udostępnia zdalne miejsce przechowywania repozytorium.

Przy rozbudowie programu poszczególne zmiany można zapisywać w oddzielnych commitach, np. dla nowej operacji drzewa. Ułatwia to śledzenie rozwoju kodu i powrót do wcześniejszej wersji. Szczegółowej historii commitów nie odtwarzano z archiwum źródeł, które nie zawiera katalogu `.git`.

Literatura

[1] Jakub Pyrdoł. Repozytorium projektu Drzewo-BST. https://github.com/pyr-jak/Drzewo-BST. Adres przekazany przez autora projektu. Opis implementacji oparto na dostarczonym archiwum źródłowym.

Spis rysunków

2.1. Drzewo przed usunięciem wartości 3 i po jej usunięciu 8

Spis tabel

2.1. Zmiana zawartości wektora po usunięciu wartości 3 8

Spis listingów

1. Dodawanie przykładowych wartości w funkcji main 4